

UNIVERSIDADE DA BEIRA INTERIOR

ENGENHARIA ELETROTÉCNICA E DE COMPUTADORES

Controlo de Semáforos com o Microcontrolador ATmega2560

Microprocessadores

Alexandre Rafael Gravelho Saraiva N^o44485

Docente: Prof. Doutor António Manuel Gonçalves Pinheiro

junho de 2025

Conteúdo

1	Lista de Acrónimos	vi
2	Resumo	viii
3	Introdução	1
4	Objectivos	2
5	Metodologia	3
5.1	Parte 1 – Sistema Básico de Semáforos	3
5.2	Parte 2 – Pedido de Peão via INT0	3
5.3	Parte 3 – Passadeira Independente via INT1	3
6	Código	4
6.1	Código WINCUPL – Parte 1 (Controlo Digital com GAL22V10)	4
6.2	Código WINCUPL – Módulo de Peões (GAL22V10)	5
6.3	Código Assembly – Parte 1 (Sequência de Semáforos)	6
6.4	Código Assembly – Parte 2 (Interrupção INT0 para pedido de peões) . . .	8
6.5	Código Assembly – Parte 2/3 (Passadeira Independente Isolada com INT1)	13
6.6	Código da Parte 3 – Semáforos com Interrupções Independentes. (Tenta- tiva sem Contadores)	18
7	Resultados e Testes	24
7.1	Testes da Parte 1 – Sequência Cíclica dos Semáforos	24
7.2	Testes da Parte 2 – Interrupção INT0 (Pedido de Peão)	25
7.3	Testes da Parte 3 – Passadeira Independente (INT1)	26
7.4	Observações Finais	27
7.5	Considerações sobre a Parte 3	27
8	Conclusão	28
9	Anexos	29

Lista de Figuras

1	Ilustração do Cruzamento com os Semáforos parte Inicial	2
2	Exemplo Semáforo no estado 5.	24
3	Exemplo da Interrupção(INT0) no ciclo dos semáforos.	25
4	Exemplo da Parte 3 no ciclo dos semáforos.	26
5	Tabela de estados dos semáforos e passadeiras em função dos sinais Q2 Q1 Q0.	29
6	PINOUT da PLD que controla os Semaforos	29
7	PINOUT da PLD que controla a luz dos os Semaforos	30
8	Slide Aula explicação parte1	31
9	Slide Aula explicação parte2	31
10	Slide Aula explicação parte3	32

1 Lista de Acrónimos

- UBI – Universidade da Beira Interior
- AVR – Alf and Vegard’s RISC Processor
- ATmega2560 – Microcontrolador da família AVR utilizado no projeto
- GPIO – General Purpose Input/Output (Entrada/Saída de Uso Geral)
- INT0 / INT1 – Interrupções externas 0 e 1 do microcontrolador
- PORTx – Registo de dados de saída (ex.: PORTA, PORTB)
- DDRx – Data Direction Register (configura pinos como entrada ou saída)
- PDx / PAx – Pinos físicos dos portos D ou A (ex.: PD0, PA2)
- LED – Light Emitting Diode (Díodo Emissor de Luz)
- PLD – Programmable Logic Device (Dispositivo Lógico Programável)
- GAL – Generic Array Logic (tipo de PLD utilizado no projeto)
- ISR – Interrupt Service Routine (Rotina de Atendimento de Interrupção)

2 Resumo

Este trabalho laboratorial consistiu na concepção e implementação de um sistema de controlo de semáforos utilizando o microcontrolador **AVR ATmega2560** e linguagem de programação **Assembly**.

O principal objetivo foi explorar mecanismos de **controlo temporal com contadores** e a gestão de eventos assíncronos através de **interrupções externas**.

A implementação foi realizada em várias fases:

- **Ciclo de Semáforos Base (1ªParte)** – Desenvolveu-se uma sequência contínua de sinais luminosos (verde, amarelo e vermelho) para um cruzamento rodoviário, com tempos definidos utilizando rotinas de atraso com base em contadores de 500 ms.
- **Pedido de Peão com INT0 (2ªParte)** – Foi incorporada a interrupção externa INT0 (ligada ao pino PD0) para simular o pedido de passagem de peões. Quando ativada, a interrupção forçava a transição imediata para o estado de vermelho para veículos e verde para peões, retornando depois à sequência normal.
- **Passadeira Independente com INT1 (3ªParte)** – Adicionou-se uma segunda interrupção externa INT1 (PD1), associada a uma passadeira autónoma. Esta funcionalidade permitia que os peões atravessassem de forma segura, sem interromper o funcionamento normal dos semáforos do cruzamento. Para garantir a segurança e evitar repetições excessivas, foi implementado um **intervalo mínimo de 30 segundos** entre ativações.

Este projeto evidenciou a importância do uso de interrupções e contadores.

3 Introdução

O presente trabalho laboratorial insere-se na unidade curricular de Microprocessadores, tendo como principal objectivo a implementação de um sistema de controlo baseado em interrupções e contadores temporizados, recorrendo ao microcontrolador *AVR ATmega2560*.

Pretende-se explorar o funcionamento e a importância das interrupções externas no controlo de sistemas com múltiplas tarefas concorrentes, assim como o uso de contadores como alternativa às tradicionais rotinas de atraso (`delay`). Este tipo de abordagem é essencial em sistemas embarcados em tempo real, onde a precisão e a resposta imediata a eventos externos são críticas.

Durante o trabalho foi desenvolvido um sistema de controlo para semáforos de veículos e de peões, onde se simula o fluxo normal de trânsito e se responde a pedidos de passagem feitos por botões físicos, utilizando as interrupções `INT0` e `INT1`. A gestão do tempo entre os estados dos semáforos é feita com base num contador de software sincronizado por um temporizador interno.

A plataforma utilizada baseia-se no microcontrolador *ATmega2560*, programado em linguagem *Assembly*, o que permitiu o contacto directo com a arquitectura do microprocessador, a manipulação dos registos e o controlo rigoroso do hardware.

4 Objectivos

O principal objectivo deste trabalho laboratorial é desenvolver um sistema de controlo de semáforos, com suporte à travessia de peões, utilizando interrupções externas e contadores para garantir a temporização precisa dos diferentes estados do sistema.

Para isso, definiram-se os seguintes objectivos específicos:

- Implementar uma sequência cíclica de estados para um sistema de semáforos num cruzamento, utilizando o microcontrolador AVR ATmega2560.
- Substituir rotinas de atraso fixas por um sistema de contagem temporizada baseado em ciclos de 500 ms, com um erro máximo permitido de ± 0.5 segundos.
- Integrar uma interrupção externa (INT0) para permitir o pedido de passagem de peões, forçando a transição segura para o vermelho nos semáforos de veículos.
- Adicionar uma segunda interrupção externa (INT1) para controlar uma passadeira independente, garantindo que os pedidos de passagem sejam aceites com um intervalo mínimo de 30 segundos entre activações.
- Assegurar o funcionamento correcto e simultâneo dos dois subsistemas (cruzamento e passadeira), demonstrando a independência e concorrência entre as interrupções.
- Promover a prática da programação em **Assembly**, reforçando o domínio sobre registos, vetores de interrupção, timers e a arquitectura interna do microcontrolador.

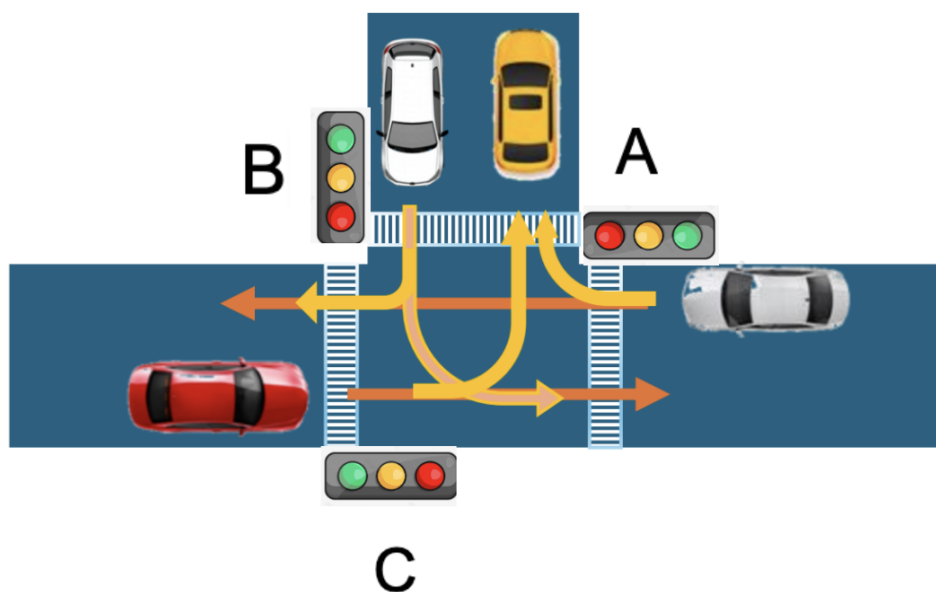


Figura 1: Ilustração do Cruzamento com os Semáforos parte Inicial

5 Metodologia

O sistema foi desenvolvido em linguagem **Assembly** para o microcontrolador **ATmega2560**, utilizando directamente os registos de configuração internos como o **EICRA** (para definir o tipo de detecção nas interrupções externas), o **EIMSK** (para a activação das interrupções) e os vetores de interrupção definidos com a directiva `.org`.

O desenvolvimento foi dividido em três partes principais, cada uma com funcionalidades específicas e crescente complexidade.

5.1 Parte 1 – Sistema Básico de Semáforos

Nesta primeira fase, foi implementado um ciclo fixo de controlo de três semáforos (vias A, B e C), com três estados principais: verde, amarelo e vermelho. O controlo dos LEDs foi feito através do envio de combinações de bits para o **PORTA**, que eram interpretadas por uma **GAL22V10** programada com expressões booleanas.

As transições entre estados foram temporizadas com rotinas de atraso baseadas em contadores, simulando períodos de 10 segundos (verde), 3 segundos (amarelo), 10(vermelho) repetidamente.

5.2 Parte 2 – Pedido de Peão via INT0

Na segunda etapa, foi configurada a interrupção externa **INT0** (pino **PD0**), activada quando um botão era pressionado. Esta interrupção permitia que o sistema desviasse temporariamente da sequência normal para dar prioridade à travessia de peões. Ao ser activada, a rotina associada forçava os semáforos a passar para o vermelho (veículos) e verde (peões), garantindo assim uma travessia segura. Após o tempo definido, o sistema regressava ao ciclo original.

5.3 Parte 3 – Passadeira Independente via INT1

Por fim, foi implementada uma passadeira independente, controlada pela interrupção **INT1** (pino **PD1**). Este subsistema era totalmente autónomo em relação ao ciclo do cruzamento principal. Quando activado, mudava os semáforos da passadeira para vermelho nos veículos e verde nos peões, permanecendo nesse estado por um tempo determinado.

Para evitar activações consecutivas muito próximas, foi incluído um contador de tempo que impunha um intervalo mínimo de 30 segundos entre duas interrupções. O controlo do tempo foi assegurado por contadores que incrementavam a cada 500 ms, permitindo assim substituir os atrasos fixos por um controlo mais rigoroso e escalável.

6 Código

6.1 Código WINCUPL – Parte 1 (Controlo Digital com GAL22V10)

```
Name      Semaforo_Partel;
PartNo     0001;
Date       2025-06-08;
Revision   1.0;
Designer   OpenAI;
Company    Projeto Escolar;
Device     g22v10;

/* === Entradas: estado enviado pelo ATmega2560
PIN 1 = Q0; /* Bit menos significativo (PA0) */
PIN 2 = Q1; /* Bit médio (PA1) */
PIN 3 = Q2; /* Bit mais significativo (PA2) */

/* === Saídas - Semáforos de veículos === */
PIN 19 = A_G; /* Semáforo A verde */
PIN 18 = A_Y; /* Semáforo A amarelo */
PIN 17 = A_R; /* Semáforo A vermelho */

PIN 16 = B_G; /* Semáforo B verde */
PIN 15 = B_Y; /* Semáforo B amarelo */
PIN 14 = B_R; /* Semáforo B vermelho */

PIN 13 = C_G; /* Semáforo C verde */
PIN 12 = C_Y; /* Semáforo C amarelo */
PIN 11 = C_R; /* Semáforo C vermelho */

/* === Saídas - Semáforos dos peões (PA, PB, PC) === */
PIN 10 = PA; /* Peão A */
PIN 9  = PB; /* Peão B */
PIN 8  = PC; /* Peão C */

/* === Lógica para semáforos de veículos === */
/* Semáforo A */
A_G = !Q2 & !Q1 & Q0; /* Estado 1 */
A_Y = !Q2 & Q1 & !Q0; /* Estado 2 */
A_R = !(A_G # A_Y); /* Estados 3,4,5,6 → Q1=1 */
```

```

/* Semáforo B */
B_G = !Q2 & Q1 & Q0;          /* Estado 3 */
B_Y = Q2 & !Q1 & !Q0;        /* Estado 4 */
B_R = !(B_G # B_Y);          /* Resto dos estados */

/* Semáforo C */
C_G = Q2 & !Q1 & Q0;          /* Estado 5 */
C_Y = Q2 & Q1 & !Q0;          /* Estado 6 */
C_R = !(C_G # C_Y);          /* Resto */

/* === Lógica para semáforos dos peões === */
PA = A_R;
PB = B_R;
PC = C_R;

```

6.2 Código WINCUPL – Módulo de Peões (GAL22V10)

```

Name      PEOESsemaforo ;
PartNo    00 ;
Date      09/06/2025 ;
Revision  01 ;
Designer  Engineer ;
Company   ubi ;
Assembly  None ;
Location  ;
Device    g22v10 ;

/* ***** INPUT PINS ***** */
PIN  2  =    Q0                ; /* */
PIN  3  =    Q1                ; /* */
PIN  4  =    Q2                ; /* */
/* ***** OUTPUT PINS ***** */
PIN  14  =    PA_G              ; /* */
PIN  15  =    PA_R              ; /* */
PIN  16  =    PB_G              ; /* */
PIN  17  =    PB_R              ; /* */
PIN  18  =    PC_G              ; /* */
PIN  20  =    PC_R              ; /* */

```

```
/* Lógica de activação para LEDs verdes dos peões */
PA_G = !Q2 & !Q1 & !Q0;
PB_G = !Q2 & !Q1 & !Q0;
PC_G = !Q2 & !Q1 & !Q0;

/* Lógica para LEDs vermelhos (complementar dos verdes) */
PA_R = !(PA_G);
PB_R = !(PB_G);
PC_R = !(PC_G);
```

6.3 Código Assembly – Parte 1 (Sequência de Semáforos)

```
.include "m2560def.inc"

jmp START          ; posição 0000H

START:
    ldi r16, LOW(RAMEND)
    out SPL, r16
    ldi r16, HIGH(RAMEND)
    out SPH, r16      ; inicialização da stack

    ldi r16, 0xFF      ; Tudo a 1s (portos como saída)
    out DDRA, R16
    out DDRB, R16

estado1:
    ldi R16, 0x01
    out PORTA, R16
    call Delay10s

estado2:
    ldi R16, 0x02
    out PORTA, R16
    call Delay3s

estado3:
    ldi R16, 0x03
```

```
    out PORTA, R16
    call Delay10s

estado4:
    ldi R16, 0x04
    out PORTA, R16
    call Delay3s

estado5:
    ldi R16, 0x05
    out PORTA, R16
    call Delay10s

estado6:
    ldi R16, 0x06
    out PORTA, R16
    call Delay3s

    jmp estado1

Delay10s:
    push R17                ; Salva R17
    ldi R17, 20             ; 20 * 0,5s = 10 segundos
loop10s:
    dec R17                 ; Decrementa contador
    brne loop10s            ; Se não for zero, repete
    pop R17                 ; Recupera R17
    ret                     ; Retorna da rotina

Delay3s:
    push R17                ; Salva R17
    ldi R17, 6              ; 6 * 0,5s = 3 segundos
loop3s:
    dec R17                 ; Decrementa contador
    brne loop3s            ; Se não for zero, repete
    pop R17                 ; Recupera R17
    ret                     ; Retorna da rotina
```

6.4 Código Assembly – Parte 2 (Interrupção INT0 para pedido de peões)

```
.include "m2560def.inc"

; Vetores de interrupção
.org 0x0000
    jmp START
.org 0x0002
    jmp INT0a      ; Interrupção INT0

; =====
; INICIALIZAÇÃO
; =====

START:
    ; Inicialização da Stack
    ldi r16, LOW(RAMEND)
    out SPL, r16
    ldi r16, HIGH(RAMEND)
    out SPH, r16

    ; Configura PORTA e PORTB como saída
    ldi r17, 0xFF
    out DDRA, r17
    out DDRB, r17

    ; Configura PORTD como entrada (INT0 = PD0)
    ldi r19, 0x00
    out DDRD, r19

    ; Configura INT0: bordo de subida
    ldi r16, 0x00
    out MCUCR, r16
    ldi r16, 0x0F
    sts EICRA, r16

    ; Ativa INT0
    ldi r16, 0x03
```

```
    out EIMSK, r16

    ; Ativa interrupções globais
    sei

    clr r19      ; r19 será a flag do pedido

; =====
; CICLO PRINCIPAL
; =====

loop:

estado1:
    ldi r16, 0x01
    out PORTA, r16
    rcall D10
    cpi r19, 0x01
    brne estado2
    rcall CheckInt
    clr r19

estado2:
    ldi r16, 0x02
    out PORTA, r16
    rcall D3

estado3:
    ldi r16, 0x03
    out PORTA, r16
    rcall D10
    cpi r19, 0x01
    brne estado4
    rcall CheckInt
    clr r19

estado4:
    ldi r16, 0x04
    out PORTA, r16
```

```
    rcall D3

estado5:
    ldi r16, 0x05
    out PORTA, r16
    rcall D10
    cpi r19, 0x01
    brne estado6
    rcall CheckInt
    clr r19

estado6:
    ldi r16, 0x06
    out PORTA, r16
    rcall D3

    rjmp loop

; =====
; INTERRUPÇÃO INTO
; =====

INT0a:
    ldi r19, 0x01      ; ativa flag de pedido de peão
    reti

; =====
; VERIFICA PEDIDO DE INTERRUPÇÃO
; =====

CheckInt:
    cpi r19, 0x01
    brne Retorna
    rcall Estado0
    clr r19
Retorna:
    ret

; =====
```

```
; ROTINA: Passagem de peão (força amarelo + vermelho)
; =====
```

Estado0:

```
    cpi r16, 0x01
    breq Forca_Estado2
    cpi r16, 0x03
    breq Forca_Estado4
    cpi r16, 0x05
    breq Forca_Estado6
    clr r19
    rcall estado1
```

Forca_Estado2:

```
    ldi r20, 0x02
    out PORTA, r20
    rcall D3
    ldi r20, 0x00
    out PORTA, r20
    rcall D5
    clr r19
    rcall estado3
```

Forca_Estado4:

```
    ldi r20, 0x04
    out PORTA, r20
    rcall D3
    ldi r20, 0x00
    out PORTA, r20
    rcall D5
    clr r19
    rcall estado5
```

Forca_Estado6:

```
    ldi r20, 0x06
    out PORTA, r20
    rcall D3
    ldi r20, 0x00
    out PORTA, r20
```

```
        rcall D5
        clr r19
        rjmp loop

; =====
; ROTINAS DE ATRASO
; =====

D10:
        ldi r18, 20
D10_Loop:
        rcall Delay500
        dec r18
        brne D10_Loop
        ret

D5:
        ldi r18, 10
D5_Loop:
        rcall Delay500
        dec r18
        brne D5_Loop
        ret

D3:
        ldi r18, 6
D3_Loop:
        rcall Delay500
        dec r18
        brne D3_Loop
        ret

Delay500:
        push R25
        ldi R25, 40
a01dj500:
        rcall a01dr500
        dec R25
        brne a01dj500
```

```
    pop R23
    ret

a01dr500:
    push R0
    clr R0
a02dj500:
    rcall a02dr500
    dec R0
    brne a02dj500
    pop R0
    ret

a02dr500:
    push R0
    clr R0
a03dj500:
    dec R0
    brne a03dj500
    pop R0
    ret
```

6.5 Código Assembly – Parte 2/3 (Passadeira Independente Isolada com INT1)

```
.include "m2560def.inc"

; Vetores de interrupção
.org 0x0000
    jmp START
.org 0x0002
    jmp INT1a      ; Interrupção INT1

; =====
; INICIALIZAÇÃO
; =====
```

START:

```
; Inicialização da Stack
ldi r16, LOW(RAMEND)
out SPL, r16
ldi r16, HIGH(RAMEND)
out SPH, r16

; Configura PORTA, PORTB e PORTC como saída
ldi r17, 0xFF
ldi r21, 0xFF
ldi r22, 0xFF
out DDRA, r17      ; parte2
out DDRB, r21      ; semáforo da passadeira
out DDRC, r22      ; luzes dos peões

; Configura PORTD como entrada (INT1 = PD1)
ldi r19, 0x00
out DDRD, r19

; Configura INTO e INT1 para bordo de subida
ldi r16, 0x00
out MCUCR, r16
ldi r16, 0x0F
sts EICRA, r16

; Ativa INTO e INT1
ldi r16, 0x03
out EIMSK, r16

; Ativa interrupções globais
sei

clr r19      ; r19 será a flag do pedido
```

verde:

```
ldi r21, 0x02      ; Verde carros
out PORTB, r21
ldi r22, 0x01      ; Vermelho peões
out PORTC, r22
```

```
    cpi r19, 0x02
    brne verde
    rcall CheckInt1
    clr r19

; =====
; INTERRUPTÇÃO INT1
; =====

INT1a:
    ldi r19, 0x02          ; Pedido de peão (INT1)
    reti

; =====
; VERIFICAÇÃO DO PEDIDO
; =====

CheckInt1:
    cpi r19, 0x02
    brne verde
    clr r19
    rcall Passagem

; =====
; ROTINA DE PASSAGEM PEÕES
; =====

Passagem:
    ; Amarelo carros
    ldi r21, 0x04
    out PORTB, r21
    ldi r22, 0x01
    out PORTC, r22
    rcall D3

    ; Vermelho carros / Verde peões
    ldi r21, 0x08
    out PORTB, r21
    ldi r22, 0x02
```

```
    out PORTC, r22
    rcall D5

    ; Volta ao estado normal (carros a verde, peões vermelho)
    ldi r21, 0x02
    out PORTB, r21
    ldi r22, 0x01
    out PORTC, r22
    rcall D10
    rcall D10
    rcall D10
    clr r19
    rcall verde

; =====
; ROTINAS DE ATRASO
; =====

D10:
    ldi r18, 20
D10_Loop:
    rcall Delay500
    dec r18
    brne D10_Loop
    ret

D5:
    ldi r18, 10
D5_Loop:
    rcall Delay500
    dec r18
    brne D5_Loop
    ret

D3:
    ldi r18, 6
D3_Loop:
    rcall Delay500
    dec r18
```

```
    brne D3_Loop  
    ret
```

Delay500:

```
    push R25  
    ldi R25, 40
```

a01dj500:

```
    rcall a01dr500  
    dec R25  
    brne a01dj500  
    pop R23  
    ret
```

a01dr500:

```
    push R0  
    clr R0
```

a02dj500:

```
    rcall a02dr500  
    dec R0  
    brne a02dj500  
    pop R0  
    ret
```

a02dr500:

```
    push R0  
    clr R0
```

a03dj500:

```
    dec R0  
    brne a03dj500  
    pop R0  
    ret
```

6.6 Código da Parte 3 – Semáforos com Interrupções Independentes. (Tentativa sem Contadores)

```
.include "m2560def.inc"

; Vetores de interrupção
.org 0x0000
    jmp START
.org 0x0002
    jmp INT0a      ; Interrupção INT0
.org 0x0004
    jmp INT1a      ; Interrupção INT1

; =====
; INICIALIZAÇÃO
; =====

START:
    ldi r16, LOW(RAMEND)
    out SPL, r16
    ldi r16, HIGH(RAMEND)
    out SPH, r16

    ldi r17, 0xFF
    ldi r21, 0xFF
    ldi r22, 0xFF
    out DDRA, r17
    out DDRB, r21
    out DDRC, r22

    ldi r19, 0x00
    out DDRD, r19

    ldi r16, 0x00
    out MCUCR, r16
    ldi r16, 0x0F
    sts EICRA, r16

    ldi r16, 0x03
```

```
        out EIMSK, r16

        sei

; =====
; CICLO PRINCIPAL
; =====

loop:
verde:
    ldi r21, 0x02
    out PORTB, r21
    ldi r22, 0x01
    out PORTC, r22
    cpi r19, 0x02
    brne verde
    rcall CheckInt1
    clr r19

estado1:
    ldi r16, 0x01
    out PORTA, r16
    rcall D10

estado2:
    ldi r16, 0x02
    out PORTA, r16
    rcall D3

estado3:
    ldi r16, 0x03
    out PORTA, r16
    rcall D10

estado4:
    ldi r16, 0x04
    out PORTA, r16
    rcall D3
```

estado5:

```
    ldi r16, 0x05
    out PORTA, r16
    rcall D10
```

estado6:

```
    ldi r16, 0x06
    out PORTA, r16
    rcall D3
```

```
    rjmp loop
```

```
; =====
; INTERRUPÇÃO INT1
; =====
```

INT1a:

```
    ldi r19, 0x02
    recall CheckInt1
    reti
```

CheckInt1:

```
    cpi r19, 0x02
    brne verde
    clr r19
    rcall Passagem
```

Passagem:

```
    ldi r21, 0x04
    out PORTB, r21
    ldi r22, 0x01
    out PORTC, r22
    rcall D3
```

```
    ldi r21, 0x08
    out PORTB, r21
    ldi r22, 0x02
    out PORTC, r22
    rcall D5
```

```
ldi r21, 0x02
out PORTB, r21
ldi r22, 0x01
out PORTC, r22
rcall D10
rcall D10
rcall D10
clr r19
rcall verde
```

```
; =====
; INTERRUPÇÃO INTO
; =====
```

INT0a:

```
cpi r16, 0x01
breq Forca_Estado2
cpi r16, 0x03
breq Forca_Estado4
cpi r16, 0x05
breq Forca_Estado6
rcall estado1
reti
```

Forca_Estado2:

```
ldi r20, 0x02
out PORTA, r20
rcall D3
ldi r20, 0x00
out PORTA, r20
rcall D5
rcall estado3
```

Forca_Estado4:

```
ldi r20, 0x04
```

```
    out PORTA, r20
    rcall D3
    ldi r20, 0x00
    out PORTA, r20
    rcall D5
    rcall estado5
```

Forca_Estado6:

```
    ldi r20, 0x06
    out PORTA, r20
    rcall D3
    ldi r20, 0x00
    out PORTA, r20
    rcall D5
```

```
; =====
; ROTINAS DE ATRASO
; =====
```

D10:

```
    ldi r18, 20
```

D10_Loop:

```
    rcall Delay500
    dec r18
    brne D10_Loop
    ret
```

D5:

```
    ldi r18, 10
```

D5_Loop:

```
    rcall Delay500
    dec r18
    brne D5_Loop
    ret
```

D3:

```
    ldi r18, 6
```

```
D3_Loop:
    rcall Delay500
    dec r18
    brne D3_Loop
    ret
```

```
Delay500:
    push R25
    ldi R25,40
```

```
a01dj500:
    rcall a01dr500
    dec R25
    brne a01dj500
    pop R23
    ret
```

```
a01dr500:
    push R0
    clr R0
```

```
a02dj500:
    rcall a02dr500
    dec R0
    brne a02dj500
    pop R0
    ret
```

```
a02dr500:
    push R0
    clr R0
```

```
a03dj500:
    dec R0
    brne a03dj500
    pop R0
    ret
```

7 Resultados e Testes

Nesta secção apresentam-se os resultados obtidos durante a fase de testes do sistema de controlo de semáforos implementado em Assembly no microcontrolador AVR ATmega2560.

7.1 Testes da Parte 1 – Sequência Cíclica dos Semáforos

O sistema básico foi testado com sucesso, verificando-se a sequência de estados configurada para os três semáforos (A, B e C). Cada estado foi visualizado através da saída no PORTA, com transições temporizadas de acordo com os valores definidos (10 segundos para verde, 3 segundos para amarelo).

As transições ocorreram conforme a tabela de estados, e os LEDs ligados aos pinos do microcontrolador indicaram corretamente os estados:

- Estado 1: Semáforo A verde, os restantes em vermelho.
- Estado 2: Semáforo A amarelo.
- Estado 3: Semáforo B verde.
- Estado 4: Semáforo B amarelo.
- Estado 5: Semáforo C verde.
- Estado 6: Semáforo C amarelo.

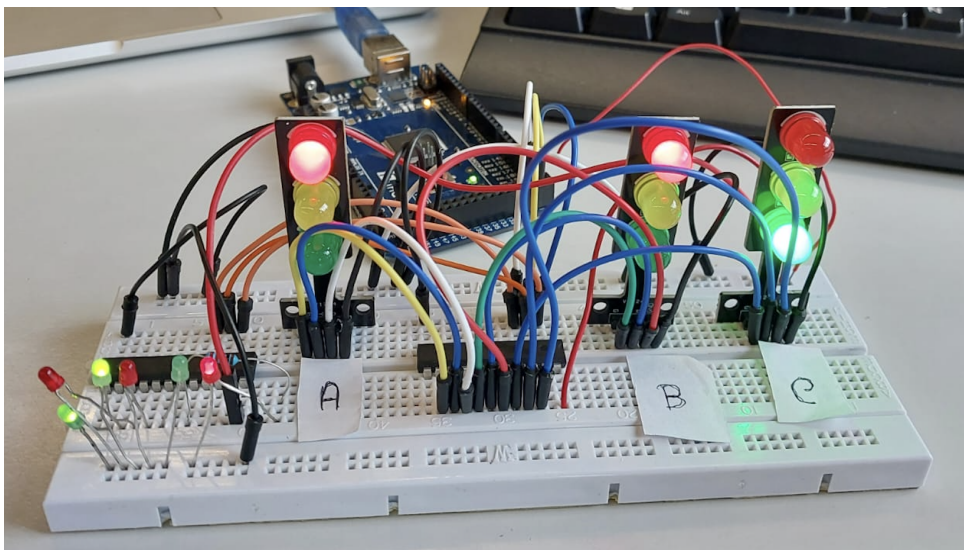


Figura 2: Exemplo Semáforo no estado 5.

7.2 Testes da Parte 2 – Interrupção INT0 (Pedido de Peão)

Foi introduzida a interrupção externa INT0 (ligada ao pino PD0) como forma de pedido de passagem de peão. Verificou-se que:

- O pedido é aceite apenas quando o semáforo está em verde.
- O sistema força a transição para o estado seguinte, com período de amarelo e todos os semáforos a vermelho antes de retomar o ciclo.
- O controlo é feito através da flag r19, com tratamento na sub-rotina `CheckInt`.

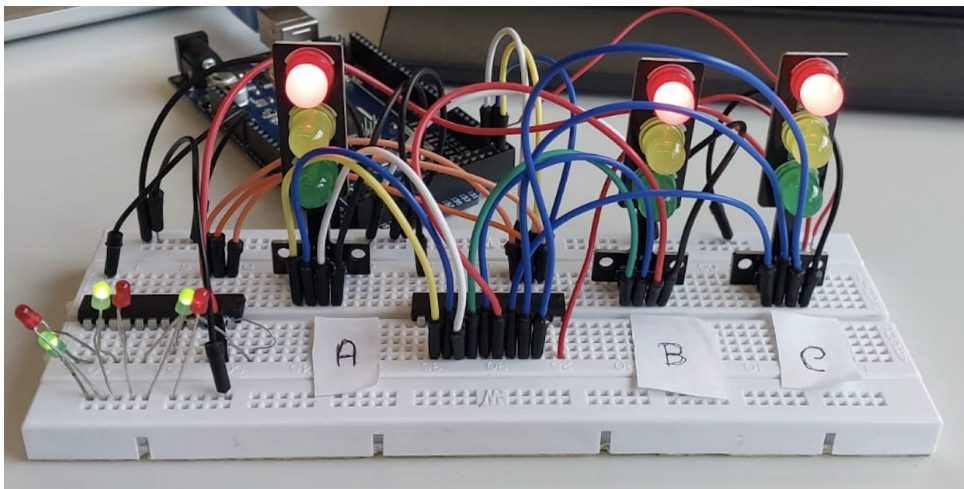


Figura 3: Exemplo da Interrupção(INT0) no ciclo dos semáforos.

7.3 Testes da Parte 3 – Passadeira Independente (INT1)

A interrupção INT1 (ligada ao pino PD1) foi usada para ativar uma passadeira independente. O comportamento esperado foi confirmado:

- Quando a interrupção é gerada, os semáforos dos veículos passam a amarelo, depois vermelho.
- Em simultâneo, os semáforos dos peões (PORTC) passam a verde.
- Após 5 segundos, os peões voltam a vermelho e os veículos retomam o estado inicial.

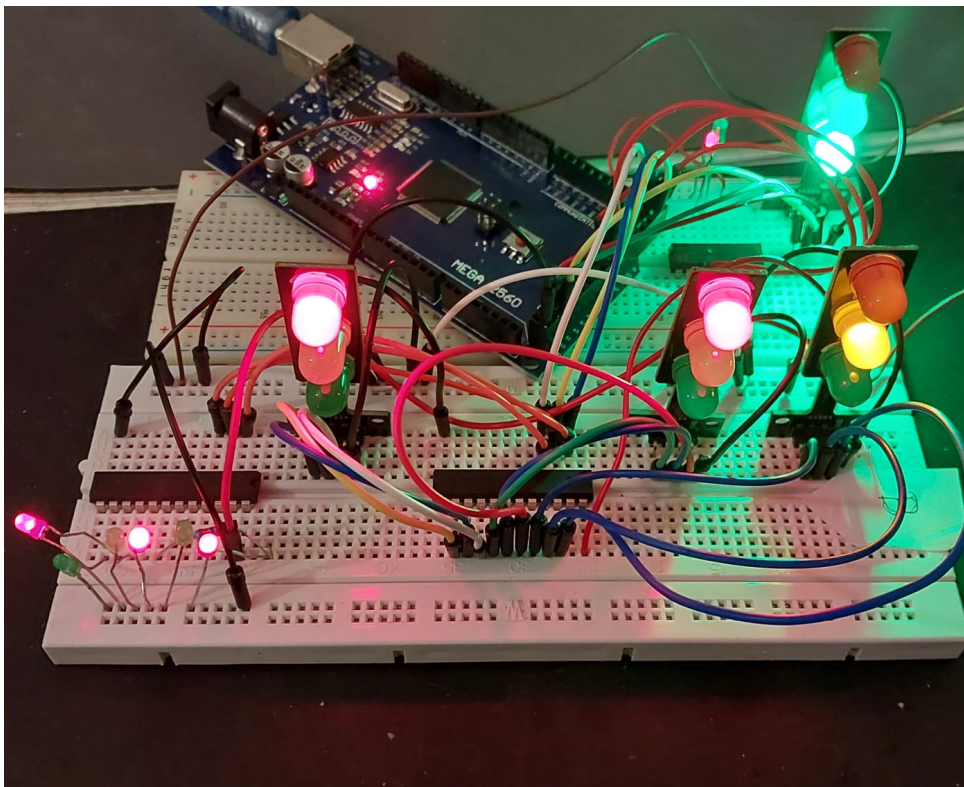


Figura 4: Exemplo da Parte 3 no ciclo dos semáforos.

7.4 Observações Finais

Embora o sistema de controlo de semáforos tenha funcionado conforme o esperado nas principais funcionalidades, a Parte 3 apresentou limitações. Após o acionamento inicial das interrupções externas (INT0 ou INT1), o sistema deixa de reconhecer novos pedidos. Este comportamento sugere uma falha na reinicialização das flags de controlo ou na gestão do fluxo após o atendimento da interrupção. Para resolver a questão, será necessária uma reorganização do código.

7.5 Considerações sobre a Parte 3

O sistema apresentou um funcionamento geral conforme o esperado.

Contudo, a implementação da Parte 3 ainda não se encontra totalmente finalizada.

Verificou-se que, ao acionar a interrupção INT0 que o ciclo sequencial dos semáforos (A, B, C) é corretamente interrompido, passando todos os semáforos a vermelho como previsto. De forma semelhante, ao acionar a interrupção INT1, a passadeira independente entra em funcionamento: o semáforo correspondente aos veículos é forçado a vermelho, enquanto o semáforo para os peões fica verde.

No entanto, após estas interrupções iniciais, o sistema deixa de responder a novos pedidos. Quando se volta a acionar fisicamente os interruptores ligados aos pinos PD0 (INT0) e PD1 (INT1) do microcontrolador, as interrupções já não são processadas como seria esperado.

Esta limitação indica que poderá estar a faltar uma reinicialização adequada das flags ou um controlo mais robusto do fluxo principal após o atendimento da interrupção. O código precisa de ser revisto de modo a garantir que, após o atendimento de cada interrupção, o sistema regressa ao estado em que está novamente preparado para reconhecer e tratar novas ativações de INT0 e INT1.

O próximo passo poderá incluir a implementação de um temporizador real (com contadores ou `Timer/Counter`) para garantir maior precisão e controlo sobre os tempos mínimos de espera.

8 Conclusão

Este trabalho permitiu consolidar, de forma prática, os conhecimentos adquiridos sobre o funcionamento de interrupções externas no microcontrolador ATmega2560, bem como a gestão do controlo de fluxo utilizando a linguagem Assembly.

Foram implementadas diversas funcionalidades, incluindo a sequência automática de estados para um sistema de semáforos e a resposta a pedidos externos através de interrupções. Adicionalmente, explorou-se o uso de temporizações com rotinas de atraso, aproximando-se a lógica implementada à de um sistema real de controlo de tráfego.

Apesar de algumas limitações na implementação da Parte 3, nomeadamente na reativação das interrupções após a sua primeira execução, o projeto permitiu compreender os principais conceitos e desafios associados ao desenvolvimento de sistemas de controlo baseados em microcontroladores e programação de baixo nível.

9 Anexos

Q2 Q1 Q0	Semáforo A	Semáforo B	Semáforo C	PA	PB	PC	Tempo
0 0 1	Verde	Vermelho	Vermelho	Vermelho	Verde	Verde	10s
0 1 0	Amarelo	Vermelho	Vermelho	Vermelho	Verde	Verde	3s
0 1 1	Vermelho	Verde	Vermelho	Verde	Vermelho	Verde	10s
1 0 0	Vermelho	Amarelo	Vermelho	Verde	Vermelho	Verde	3s
1 0 1	Vermelho	Vermelho	Verde	Verde	Verde	Vermelho	10s
1 1 0	Vermelho	Vermelho	Amarelo	Verde	Verde	Vermelho	3s

Figura 5: Tabela de estados dos semáforos e passadeiras em função dos sinais Q2 Q1 Q0.

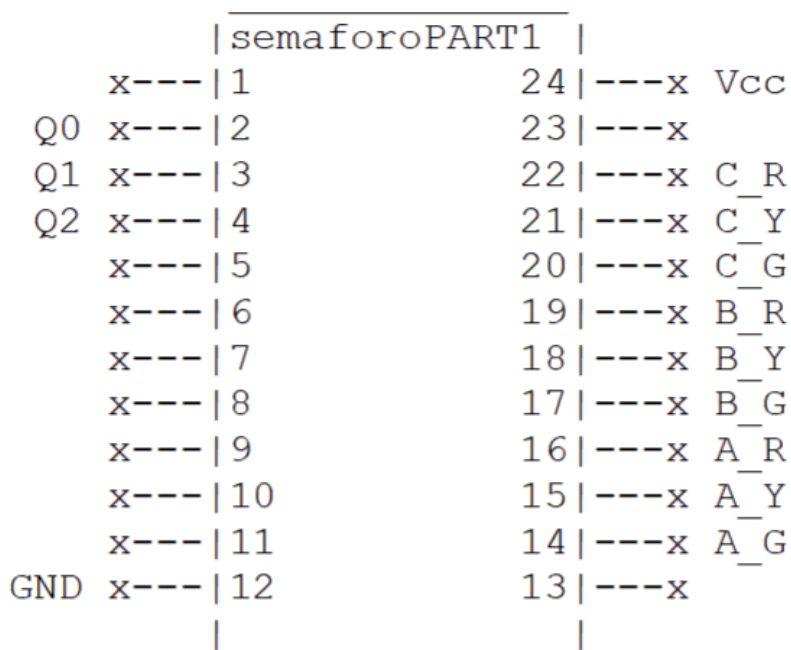


Figura 6: PINOUT da PLD que controla os Semaforos

		PEOESsemaforo	
	x---	1	24 ---x Vcc
Q0	x---	2	23 ---x
Q1	x---	3	22 ---x
Q2	x---	4	21 ---x
	x---	5	20 ---x PC_R
	x---	6	19 ---x
	x---	7	18 ---x PC_G
	x---	8	17 ---x PB_R
	x---	9	16 ---x PB_G
	x---	10	15 ---x PA_R
	x---	11	14 ---x PA_G
GND	x---	12	13 ---x

Figura 7: PINOUT da PLD que controla a luz dos os Semaforos

Parte 1

- Controlar um sistema de semáforos para o cruzamento da figura.
- O sinal verde dos veículos deve ser colocado 10 segundos.
- O sinal amarelo dos veículos deve ser colocado 3 segundos.
- O sinal verde dos peões deve ser colocado 5 segundos.
- Todas as outras opções ficam ao vosso critério.

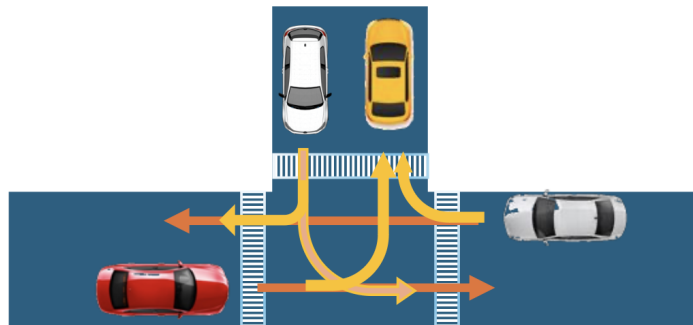


Figura 8: Slide Aula explicação parte1

Parte 2

- Considere em adição à Parte 1 que os semáforos dos peões só são considerados na sequência temporal quando houver uma solicitação.
- A solicitação deve ser feita via a ativação de uma interrupção (INT0).

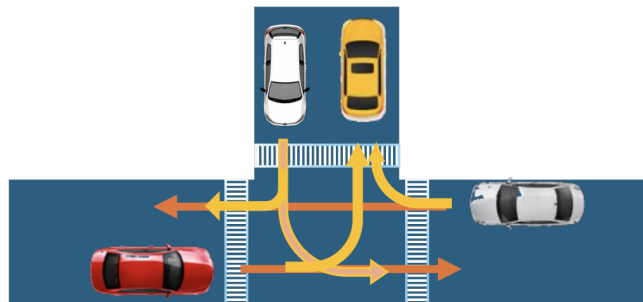


Figura 9: Slide Aula explicação parte2

Parte 3

- Considere que se adiciona uma passadeira controlada por semáforos, com funcionamento independente (ver figura). Nesta passadeira deve ser colocado o semáforo verde para os peões quando solicitado via a ativação de uma interrupção (INT1).
- O intervalo mínimo entre duas passagens do peão a verde deve ser de 30 segundos.
- O sistema de semáforos do cruzamento deve continuar o seu funcionamento normal, independente da passadeira.
- É tolerado um erro máximo nos tempos do funcionamento normal de 0.5 segundos no máximo.



Figura 10: Slide Aula explicação parte3